

RFC-0002

Modelo de Autorização Evolutivo — RBAC + ABAC com Extensão para ReBAC

Status: Aprovado e Atualizado (v1.1)

Autor: Equipe de Arquitetura

Data: 07/05/2026

Depende de: RFC-0001, RFC-0005, Arquitetura de Identidade e Acesso Segura

1. Contexto

O sistema é multi-tenant, com usuários podendo pertencer a múltiplas organizações simultaneamente. A autorização não pode depender exclusivamente de papéis (roles) do provedor de identidade (Keycloak), e deve operar sob o paradigma de *Zero Trust* (Confiança Zero), garantindo que a credencial técnica seja convertida em uma identidade canônica real antes de qualquer concessão de acesso.

Objetivo: Estabelecer um modelo de autorização:

- Desacoplado das regras de negócio do IdP.
 - Multi-tenant aware.
 - Protegido contra vazamento de dados sensíveis em Tokens (Lightweight Tokens).
 - Evolutivo (RBAC → ABAC → ReBAC).
 - Auditável e aderente à LGPD.
-

2. Princípios Arquiteturais

1. **Autenticação ≠ Autorização:** Keycloak autentica a conta técnica; o Backend Spring Boot resolve quem é a pessoa física e autoriza suas ações.
 2. **Autorização resolvida no domínio:** Sempre contextual por organização (`organizacao_id`).
 3. **Negação por padrão (deny-by-default).**
 4. **Fetch On-Demand:** Decisões baseadas em atributos (ABAC) nunca confiam no payload do JWT, mas sim em consultas em tempo real ao banco de dados.
 5. **Log de decisão obrigatório:** Toda barreira de acesso gera rastro na trilha de auditoria.
-

3. Modelo Híbrido Proposto

A arquitetura de autorização é composta por três camadas sobrepostas:

- **Camada 1 — RBAC (base estrutural):** Define papéis organizacionais dentro do tenant.
 - **Camada 2 — ABAC (controle fino):** Define regras dinâmicas baseadas em atributos físicos, biométricos e temporais.
 - **Camada 3 — ReBAC (implementado):** Permite autorizações baseadas em relacionamentos hierárquicos e delegações através de grafos.
-

4. RBAC (Organizational Role-Based Access Control)

4.1 Papéis Base no MVP

- OWNER
- ADMIN
- MANAGER
- PARTICIPANT
- AUDITOR (preparação futura)

Os papéis são armazenados estritamente na tabela **folha_ponto.vinculo_usuario**. Não devem residir no IdP (Keycloak).

4.2 Escopo do Papel

A Role é sempre contextual: (usuario_id, organizacao_id) → papel. Nunca deve ser tratada de forma global (cross-tenant).

5. ABAC (Attribute-Based Access Control) e Lightweight Tokens

A camada ABAC é vital para evitar a explosão de roles e suportar regras dinâmicas inerentes à biometria e presença.

5.1 O Padrão Lightweight Token (Diretriz de Segurança)

Para estrita conformidade com a LGPD, o Token JWT gerado pelo Keycloak será **desprovido de qualquer dado sensível** (sem CPF, sem biometria, sem endereço e sem roles de negócio). Ele carrega apenas a claim Sub (Subject). **Todos os atributos abaixo são extraídos em tempo real (Fetch On-Demand) através de queries otimizadas no PostgreSQL.**

5.2 Atributos Relevantes para a Policy Engine

Subject (usuário avaliado no banco)

- papel (via tabela folha_ponto.vinculo_usuario)
- status (ativo/suspenso via tabela folha_ponto.vinculo_usuario)
- Existência de assinatura na folha_ponto.perfil_biometrico
- nivel_confianca (via tabela folha_ponto.dispositivo)

Resource (recurso/evento)

- `organizacao_id` e `unidade_id`
- `status_tecnico` e `status_administrativo` (via tabela `folha_ponto.registro_presenca`)

Environment (ambiente)

- `recebido_no_servidor_em` (horário oficial blindado)
 - IP address e localização geográfica (via PostGIS na tabela `folha_ponto.unidade`)
 - `modo_registro`
-

6. Estrutura Técnica da Autorização (Spring Security)

A ponte entre a Identidade Técnica e a Autorização de Negócio ocorre dentro do ecossistema do Spring Security.

6.1 O Conversor de Identidade (`customJwtAuthenticationConverter`)

Antes de qualquer Controller ser atingido, o Spring Security intercepta a requisição e invoca um conversor customizado (`customJwtAuthenticationConverter`). O papel deste componente é crítico:

1. Extrai o `sub` do JWT.
2. Consulta a camada de dados de identidade para descobrir qual Pessoa Física (CPF) é a legítima dona daquele `sub`.
3. Busca na tabela `folha_ponto.vinculo_usuario` as *Roles* ativas para a organização alvo.
4. Converte a abstração de credencial em um **`BusinessPrincipalToken`**, enriquecido com o CPF e as *GrantedAuthorities* exclusivas do banco local.

6.2 Camada de Policy Engine Interna

Com o `BusinessPrincipalToken` injetado no contexto, o motor de políticas é acionado:

Interface:

```
AuthorizationService.evaluate(  
    businessPrincipal,  
    action,  
    resource,  
    environment  
) -> Decision
```

Decision:

- `PERMIT`
 - `DENY`
 - `PERMIT_WITH_CONDITIONS`
-

7. Integração com Keycloak

Uso **restrito** a:

- Autenticação técnica e federação (AD/LDAP).
- Emissão de Lightweight Token OIDC.
- Identificação imutável da conta técnica (sub).

Não usar:

- Role organizacional embarcada no token.
- Tenant fixo no token.

Motivação para evitar:

- Token inflation (estouro de limite de cabeçalho HTTP).
 - Inconsistência organizacional (usuário acessando com token defasado).
 - Espalhamento indevido de PII (Personally Identifiable Information) e quebra da LGPD.
-

8. Multi-Tenant Enforcement

Toda policy deve validar obrigatoriamente a fronteira do tenant através da relação entre o recurso e o vínculo do usuário identificado no `BusinessPrincipalToken`:

```
recurso.organizacao_id == vinculo_usuario.organizacao_id
```

Qualquer divergência relacional resultará em **DENY** imediato (HTTP 403 Forbidden).

9. Auditoria de Decisão

Toda decisão sensível deve gerar registro imutável na tabela **`folha_ponto.log_auditoria`**:

- `usuario_id`
 - `organizacao_id`
 - `acao` (método/endpoint)
 - `entidade_id` (recurso)
 - `decisao`
 - `criado_em` (timestamp em UTC)
-

10. ReBAC (Relacionamentos Complexos)

O modelo relacional suporta autorizações baseadas em grafos e vínculos diretos, contornando limitações clássicas do AD/LDAP.

Cenários suportados:

- Delegação temporária (Gestor Substituto).

- Supervisor indireto.
- Hierarquia estrutural entre unidades.

Estrutura Física: O motor de políticas consultará a tabela já implementada **folha_ponto.relatorio**, cruzando `sujeito_id` e `objeto_id` dentro do período de vigência (`inicio_validade` / `fim_validade`). Isso isenta a equipe de TI de criar grupos temporários no Active Directory para gerir substituições de férias.

11. Anti-Patterns a Evitar

- Hardcode de role diretamente no controller.
 - Lógica condicional de autorização espalhada pelo código (ao invés de isolada no Policy Engine).
 - Embutir o CPF ou validações biométricas no Payload do JWT.
 - Confiar a decisão de acesso a validações puramente do lado do React (Frontend).
-

12. Roadmap Evolutivo

- **Fase 1 — MVP:** Conversão do sub para Identidade Canônica, RBAC simples, e enforcement obrigatório de `organizacao_id`.
 - **Fase 2:** Introdução do ABAC consultando atributos em tempo real no banco.
 - **Fase 3:** Externalização parcial de regras lógicas via `condicao_json` na tabela `folha_ponto.excecao_permissao_usuario`.
 - **Fase 4:** ReBAC maduro, consumindo a tabela `relacionamento` para grafos de poder.
-

13. Exemplo de Fluxo de Autorização Completo (Registro de Presença)

1. **Acesso:** O usuário autenticado dispara o evento bruto de presença no SPA React, anexando o *Lightweight Token* na requisição.
2. **Interceptação Técnica:** O Spring Security intercepta a chamada e valida matematicamente a assinatura do JWT baixando as chaves públicas (JWKS) do Keycloak.
3. **Resolução de Identidade (A Dica de Ouro):** O `customJwtAuthenticationConverter` entra em ação. Ele extrai o `sub` do token, consulta o banco de dados e localiza a Identidade Canônica (a Pessoa Física dona daquele `sub`).
4. **Resolução de Vínculo:** O conversor busca na tabela `folha_ponto.vinculo_usuario` os papéis da pessoa especificamente para a Organização daquela rota.
5. **Contexto de Segurança:** Um `BusinessPrincipalToken` (agora ciente do CPF e das Roles locais) é injetado no `SecurityContextHolder`.
6. **Policy Evaluation:** A anotação (ex: `@PreAuthorize`) invoca o motor de políticas, que

- avalia os atributos ABAC (precisão do GPS, validade biométrica na base local).
7. **Decisão:** Se **PERMIT**, a camada de serviço persiste o evento. Se **DENY**, a execução é interrompida com HTTP 403 Forbidden.
 8. **Trilha:** A ação e a decisão são gravadas de forma imutável no `log_auditoria`.
-

14. Conclusão Estratégica

A adoção do modelo de **Resolução Tardia de Autorização** transforma o sistema numa fortaleza contra falhas externas. Se o Keycloak for recriado ou se o servidor trocar seu e-mail do LDAP pelo login do Google, a sua Identidade Canônica e todas as suas autorizações organizacionais no banco do Spring Boot permanecerão intactas.

A separação inegociável entre *Autenticação (IAM)* e *Autorização (Backend)* é o que garante a longevidade, flexibilidade e a total blindagem aos requisitos de proteção de dados sensíveis estabelecidos pela LGPD.